```
Output:
int(0)
int(0)
```

The problem here, of course, is that the string literal `"\u{11F}"` is UTF-8 (assuming a default encoding of UTF-8). At this point we could choose to use `mb_convert_encoding()` so our strings are identically encoded…

```
var_dump(mb_substr_count(
    $utf32Str,
    mb_convert_encoding("\u{11F}ari", 'UTF-32')
));
```

```
Output:
int(1)
```

… but why bother? If we know (or can determine in code) the encoding of the strings, we might as well just use `substr_count()` after the necessary conversion(s). As long as both strings are encoded correctly, then all we need is the byte-level comparison of `substr_count()`.

```
var_dump(substr_count(
    mb_convert_encoding($utf32Str, 'UTF-8', 'UTF-32'),
    "\u{11F}"
));
```

```
Output:
int(1)
```

Note that `mb_substr_count()` does not offer the length and offset parameters of `substr_count()`, so a solution like the one above actually offers *more* utility than its `mbstring` counterpart. The `mbstring` extension has a few gotchas like this, and some tools (of varying quality) for dealing with them. There are also alternative extensions for handling multi-byte data. We'll look at them in more detail later in the chapter.

Phew! That was quite a lot to take in, wasn't it? It's inevitable, I'm afraid. PHP exists to crunch text because that's what the internet did back in the day (it wasn't all funny pictures of cats and TikTok in the 90's). Well done for getting this far; treat yourself to a cup of tea, and we'll get on with the next category…